

SQL Notes4 (5/22/2026)

Cardinality:

Cardinality describes the relationship between rows in different tables, specifically, how many rows in one table can relate to rows in another.

One-to-One (1:1)

One row in Table A matches exactly one row in Table B.

Ex: A person has one passport, and each passport belongs to one person.

One-to-Many (1:N)

One row in Table A can match many rows in Table B. This is the most common relationship.

Ex: One customer can place many orders.

Many-to-One (N:1)

This is simply the reverse perspective of one-to-many.

Ex: Many employees work in one department.

Many-to-Many (M:N)

Many rows in Table A can relate to many rows in Table B.

Ex: Students can enroll in many courses and courses can have many students.

Joins:

A JOIN is used to combine rows from two or more tables based on a related column between them.

Inner join:

Returns only matching rows from both tables.

```
SELECT Customers.name, Orders.order_id  
FROM Customers  
INNER JOIN Orders  
ON Customers.customer_id = Orders.customer_id;
```

LEFT JOIN:

Returns: All rows from the left table matching rows from the right table. NULL if no match exists.

```
SELECT Customers.name, Orders.order_id  
FROM Customers  
LEFT JOIN Orders  
ON Customers.customer_id = Orders.customer_id;
```

RIGHT JOIN:

Returns: All rows from the right table matching rows from the left table.

```
SELECT Customers.name, Orders.order_id  
FROM Customers  
RIGHT JOIN Orders  
ON Customers.customer_id = Orders.customer_id;
```

FULL OUTER JOIN:

Returns: All rows from both tables matched where possible. NULL where no match exists

```
SELECT Customers.name, Orders.order_id  
FROM Customers
```

FULL OUTER JOIN Orders

ON Customers.customer_id = Orders.customer_id;

LEFT JOIN Excluding INNER JOIN:

Returns rows present only in the left table.

SELECT c.*

FROM Customers c

LEFT JOIN Orders o

ON c.customer_id = o.customer_id

WHERE o.customer_id IS NULL;

RIGHT JOIN Excluding INNER JOIN:

Returns rows present only in the right table.

SELECT o.*

FROM Customers c

RIGHT JOIN Orders o

ON c.customer_id = o.customer_id

WHERE c.customer_id IS NULL;

FULL OUTER JOIN Excluding INNER JOIN:

Returns all non-matching rows from both tables.

SELECT *

FROM Customers c

```
FULL OUTER JOIN Orders o
ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL
      OR o.customer_id IS NULL;
```

CROSS JOIN:

Returns the Cartesian product of both tables. Every row from table A combines with every row from table B.

```
SELECT Customers.name, Products.product_name
FROM Customers
CROSS JOIN Products;
```

If: Customers = 3 rows, Products = 4 rows, Result = 12 rows.

SELF JOIN:

A table joins with itself. Useful for hierarchical data.

```
SELECT A.employee_name AS Employee,
       B.employee_name AS Manager
FROM Employees A
LEFT JOIN Employees B
ON A.manager_id = B.employee_id;
```

Common Table expressions (CTE):

A temporary result set used to simplify complex queries.

A CTE improves: Readability, modularity, complex query handling, recursion support.

Recursive CTE:

Used for: Hierarchy data, tree structures, organizational charts, generating sequences